PIXEL-8 CPU

A Learner's Guide

How the CPU I built works, the data structures and C++ concepts inside it, and exactly where each one is in the code.

Module	CPU, memory chips, Bus, I/O ports, Console (`CPU/Aashish/src`)
Author	Aashish Singh
Team	NULL Pointers · DSCPP-III-2026-T193 (Arnav Bisht, Piyush Shukla, Ashmit Sharma, Aashish Singh)
Language	C++11, built with g++ (MinGW) and `mingw32-make`, no external libraries
Tests	193 checks in `test_cpu.cpp`; 322 across all five project suites; 0 failing
Made	29 October 2026. Every code snippet was copied from the source files when this PDF was built, so the file names and line numbers match the code.

How to read this guide

- **Part 1** explains the CPU from zero: the parts, the memory map, and what happens when one instruction runs.
- **Part 2** covers the data structures from the DS syllabus: array, stack, queue, binary search and hash map, plus why there is no linked list.
- **Part 3** covers C++ and OOP: classes and objects, the four pillars, constructors, friend functions, friend classes and static members.
- **Part 4** shows how the CPU is tested. **Part 5** is a viva cheat sheet with a 60-second explanation.

Every concept follows the same pattern: what it is, **where** it is (file and line numbers), the **real code**, **why** it is used there, and a green box with words you can use when you explain it. Highlighted lines in a code box are the ones to point at.

Contents

Part 1 · How the CPU works . . . 3

1.1 The big idea . . . 3
1.2 Bytes, words and hexadecimal . . . 4
1.3 Registers and flags: what the CPU remembers . . . 4
1.4 The memory map . . . 5
1.5 The instruction set . . . 5
1.6 One instruction: fetch, decode, execute . . . 6
1.7 Worked example: 5 + 3 on the real CPU . . . 8
1.8 The ALU: how flags are set . . . 8
1.9 Faults: a bad program stops cleanly . . . 9
1.10 The Console: the whole machine in one object . . . 9

Part 2 · Data structures . . . 11

2.1 Array . . . 11
2.2 Stack . . . 12
2.3 Queue (circular) . . . 14
2.4 Binary search . . . 16
2.5 Hash map . . . 18
2.6 Linked list: deliberately not used . . . 19

Part 3 · C++ and OOP . . . 20

3.1 Classes and objects . . . 20
3.2 Encapsulation . . . 21
3.3 Abstraction . . . 21
3.4 Inheritance . . . 22
3.5 Polymorphism . . . 22
3.6 Constructors . . . 23
3.7 Friend function: operator<< . . . 24
3.8 Friend class: CpuSnapshot . . . 25
3.9 Static members . . . 26
3.10 Functions . . . 26
3.11 Other C++ features worth knowing . . . 27

Part 4 · Testing . . . 28

Part 5 · Viva cheat sheet . . . 29

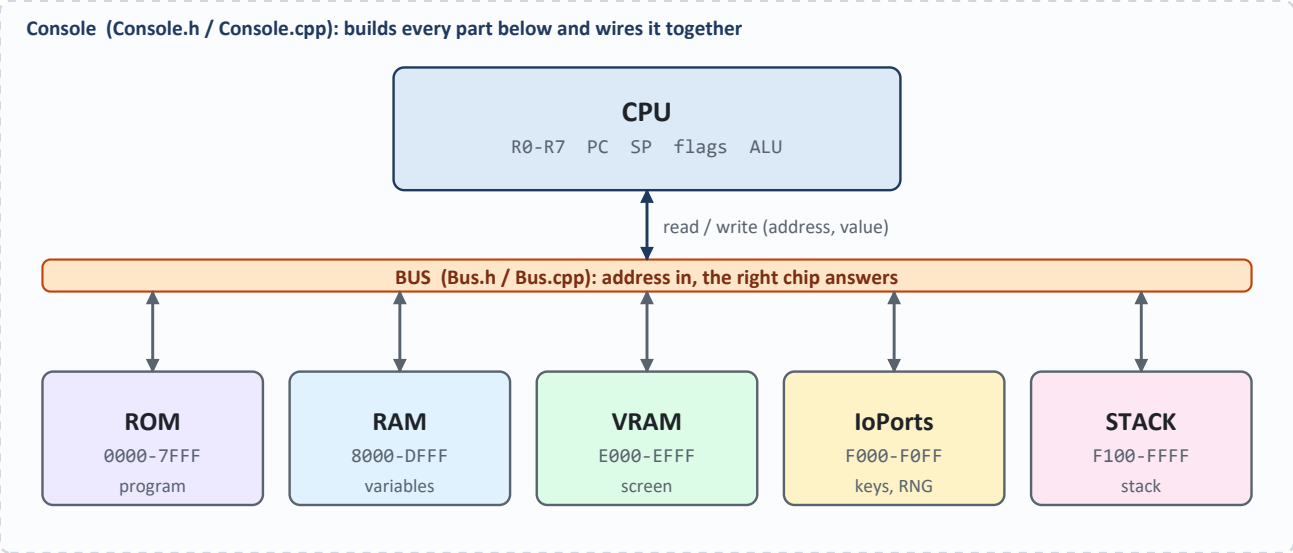
Explain the CPU in 60 seconds . . . 29
Quick answers . . . 29

Part 1 · How the CPU works

From zero: the parts, the numbers, and one instruction from start to finish.

1.1 The big idea

PIXEL-8 is an 8-bit computer that exists only in software. My module is its **hardware**: a processor, memory chips, and the wires between them, each written as a C++ class. A program is just a list of bytes. The CPU reads those bytes one instruction at a time and does what each one says: put a number in a register, add two numbers, jump somewhere else, store a pixel on the screen.



The whole machine. The CPU never talks to a chip directly: every read and write goes through the Bus, which decides which chip owns the address.

Part	Files	What it is in a real computer
`CPU`	`CPU.h`, `CPU.cpp`	the processor: registers, ALU (arithmetic), control unit (fetch-decode-execute)
`Bus`	`Bus.h`, `Bus.cpp`	the address and data wires: sends each address to the chip that owns it
`Rom`	`Rom.h`, `Rom.cpp`	read-only program memory; only `load()` can fill it
`Ram`	`Ram.h`, `Ram.cpp`	read/write memory; one class, used for three chips: RAM, VRAM and STACK
`IoPorts`	`IoPorts.h`, `IoPorts.cpp`	keyboard buttons, random numbers, frame counter, text output
`OpcodeTable`	`Opcodes.h`, `Opcodes.cpp`	the instruction dictionary: opcode byte to name, format, length, cycles
`Disassembler`	`Disassembler.h/.cpp`	turns bytes back into readable assembly, e.g. `LDI R0, 0x05`
`InstructionTrace`	`InstructionTrace.h/.cpp`	a flight recorder: the last 16 instructions (a circular queue)
`CpuSnapshot`	`CpuSnapshot.h/.cpp`	a save point for the whole CPU (a friend class)
`Console`	`Console.h`, `Console.cpp`	the whole machine: creates every chip and plugs it into the Bus
`MemoryMap`, `Fault`, `Bits`	`.h` files	address constants, the error record, byte/bit helpers

1.2 Bytes, words and hexadecimal

An 8-bit CPU works on **bytes**: 8 bits, values 0 to 255. Addresses are 16 bits wide (a **word**, 0 to 65,535), so the CPU can reach 65,536 bytes, which is 64 KB. We write numbers in hex because one hex digit is exactly 4 bits: `0xFF` = 255, `0x8000` = 32,768. Both types are **unsigned** and wrap around like real hardware: 255 + 1 = 0.

CPU/Aashish/src/Bits.h · lines 6–15

```
6   typedef uint8_t Byte;  //8 bits,  0 to 255
7   typedef uint16_t Word;  //16 bits, 0 to 65535 which is used for addreses
8
9   bool getBit(Byte value, int n);    //is bit n on?
10  Byte setBit(Byte value, int n);   //turn bit n on.
11  Byte clearBit(Byte value, int n);   //turn bit n off
12
13  Byte lowByte(Word w);    //bits 0-7 of a word
14  Byte highByte(Word w);   //bits 8-15 of a word
15  Word makeWord(Byte low, Byte high);   //join two bytes into on word
```

The helpers use the **bitwise operators** `>>`, `<<`, `&`, `|` and `~` to test, set and clear single bits and to split a word into two bytes. Flags (next section) are stored as bits, and these are the functions that read and write them:

CPU/Aashish/src/Bits.cpp · lines 5–30

```
5   bool getBit(Byte value, int n){
6       if(n < 0 || n > 7) return false;
7       return (value >> n) & 1;
8   }
9
10  Byte setBit(Byte value, int n){
11      if(n < 0 || n > 7) return value;
12      return (Byte)(value | (1 << n));
13  }
14
15  Byte clearBit(Byte value, int n){
16      if(n < 0 || n > 7) return value;
17      return (Byte)(value & ~(1 << n));
18  }
19
20  Byte lowByte(Word w){
21      return (Byte)(w & 0xFF);
22  }
23
24  Byte highByte(Word w){
25      return (Byte)((w >> 8) & 0xFF);
26  }
27
28  Word makeWord(Byte low, Byte high){
29      return (Word)(low | (high << 8));
30  }
```

1.3 Registers and flags: what the CPU remembers

Name	Size	Job
`R0`–`R7`	8 bits each	general registers: the CPU's scratch space for numbers it is working on
`P` = `R6:R7`	16 bits	R6 (high byte) and R7 (low byte) joined into one address, for `LDP`/`STP` (read/write memory at P)
`PC`	16 bits	program counter: the address of the next byte to fetch. Starts at `0x0000`
`SP`	16 bits	stack pointer: the next free stack slot. Starts at `0xFFFF` (empty stack)
`flags`	4 bits	Z (bit 0) result was zero · C (bit 1) carry out / borrow · N (bit 2) bit 7 of the result is 1 (negative) · V (bit 3) signed overflow

Name	Size	Job
`cycles`	counter	how much work has been done; each instruction adds its cycle cost

Inside the class these are plain private members. Note the register file is an **array** of 8 bytes, so `R3` is simply `reg[3]`:

CPU/Aashish/src/CPU.h · lines 54–63

```
54          Bus&  bus;
55          Byte  reg[NUM_REGS];
56          Word  pc;
57          Word  sp;
58          Byte  flags;
59          CpuState state;
60          Fault fault;
61          unsigned long cycles;
62          Word  instrStart;                    //PC where the current instruction began
63          InstructionTrace trace;
```

1.4 The memory map

Every one of the 65,536 addresses belongs to exactly one chip. The ranges touch with no gaps and never overlap (a test checks this):

Range	Size	Chip	Used for
`0x0000–0x7FFF`	32 KB	ROM	the program (read-only)
`0x8000–0xDFFF`	24 KB	RAM	variables
`0xE000–0xEFFF`	4 KB	VRAM	screen pixels: 32 × 16, one byte per pixel at `0xE000 + y*32 + x`
`0xF000–0xF0FF`	256 B	IoPorts	INPUT `F000`, RNG `F001`, PPU_CTRL `F002`, FRAME `F003`, CONSOLE_OUT `F004`
`0xF100–0xFFFF`	3.75 KB	STACK	the stack, growing **down** from `0xFFFF`

CPU/Aashish/src/MemoryMap.h · lines 7–16

```
7       const Word ROM_START    = 0x0000;
8       const Word ROM_END      = 0x7fff;       //32 KB, ,program
9       const Word RAM_START    = 0x8000;
10      const Word RAM_END      = 0xDfff;       //24 KB, variables
11      const Word VRAM_START   = 0xE000;
12      const Word VRAM_END     = 0xEFFF;       //4 KB, screen pixels
13      const Word IO_START     = 0xF000;
14      const Word IO_END       = 0xF0FF;       //256 bytes, ports
15      const Word STACK_START  = 0xF100;       //lowest address the stack may use
16      const Word STACK_END    = 0xFFFF;       // SP starts here
```

1.5 The instruction set

PIXEL-8 has **38 instructions** in 6 groups. Each instruction is 1 to 4 bytes: an **opcode** byte that says which instruction it is, then its operands.

Group	Instructions
System (2)	`NOP` do nothing · `HLT` stop
Data movement (8)	`MOV LDI` load a number · `LD ST` load/store at an address · `LDP STP` load/store at P · `INP DEP` P+1, P−1
Arithmetic and logic (11)	`ADD ADC SUB SBC AND OR XOR CMP` (register-register) · `ADDI SUBI CMPI` (register-number)
Single register (5)	`INC DEC NOT SHL SHR`

Group	Instructions
Control flow (8)	`JMP` · `JZ JNZ JC JNC JN` (jump if a flag says so) · `CALL RET`
Stack (4)	`PUSH POP` (a register) · `PUSHF POPF` (the flags)

Where the operand bytes go is fixed by the instruction's **format**. There are 7 formats:

CPU/Aashish/src/Opcodes.h · lines 29–37

```
29  enum class Format {
30      Implied,    //opcode                    1 byte   NOP, HLT, RET
31      Reg,        //opcode, reg               2 bytes  INC R1, PUSH R3
32      RegReg,     //opcode, (dst<<4 | src)    2 bytes  ADD R0, R1
33      RegImm,     //opcode, reg, value        3 bytes  LDI R0, 42
34      Abs,        //opcode, addrLow, addrHigh 3 bytes  JMP 0x0010
35      RegAbs,     //opcode, reg, low, high    4 bytes  LD R0, [0x8000]
36      RegInd      //opcode, reg (address in P) 2 bytes  LDP R0
37  };
```

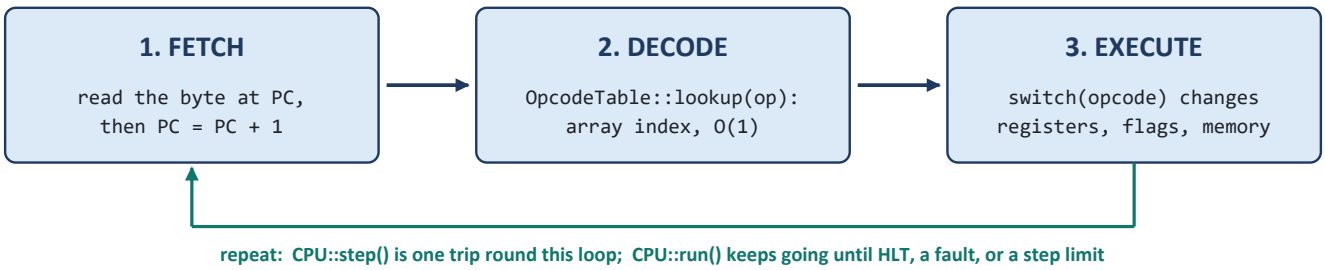
Three encodings to get a feel for it: `LDI R0, 5` is `11 00 05` (opcode, register, number). `ADD R0, R1` is `30 01`: both register numbers share one byte, destination in the high 4 bits, source in the low 4. `JMP 0x0010` is `60 10 00`: addresses are **little-endian**, low byte first.

All 38 instructions are listed once, in one array. The length is not typed in: it is worked out from the format, so the two can never disagree:

CPU/Aashish/src/Opcodes.cpp · lines 13–23

```
13      //the one list of all 38 instructions. Length is not typed here: it comes from lengthOf(format),
14      //so it can never disagree with the format.
15      //cycles = bytes fetched + extra memory reads/writes the instruction makes
16      const Entry ENTRIES[] = {
17          {OP_NOP,   "NOP",   Format::Implied, 1},
18          {OP_HLT,   "HLT",   Format::Implied, 1},
19
20          {OP_MOV,   "MOV",   Format::RegReg,  2},
21          {OP_LDI,   "LDI",   Format::RegImm,  3},
22          {OP_LD,    "LD",    Format::RegAbs,  5},
23          {OP_ST,    "ST",    Format::RegAbs,  5},
```

1.6 One instruction: fetch, decode, execute



The cycle every CPU runs. One call to `CPU::step()` is one trip round it.

CPU/Aashish/src/CPU.cpp · lines 37–63

```
37  StepResult CPU::step(){
38      if(state == CpuState::Halted) return StepResult::Halt;
39      if(state == CpuState::Faulted) return StepResult::Fault;
40
41      bus.clearFault();                   //only faults caused by THIS instruction count
42      instrStart = pc;
43
44      //FETCH
45      Byte opcode = fetchByte();
46      if(busFailed()) return StepResult::Fault;
47      trace.enqueue(instrStart, opcode);      //before decoding, so a bad opcode is recorded too
48
49      //DECODE
50      const OpcodeInfo& info = OpcodeTable::lookup(opcode);
51      if(!info.valid){
52          raise(FaultType::InvalidOpcode, instrStart, "Invalid opcode " + hex8(opcode));
53          return StepResult::Fault;
54      }
55
56      //EXECUTE
57      execute(opcode, info);
58      if(state == CpuState::Faulted) return StepResult::Fault;
59
60      cycles += info.cycles;
61      if(state == CpuState::Halted) return StepResult::Halt;
62      return StepResult::Ok;
63  }
```

- **Fetch:** read the byte at PC, then move PC on by one (`fetchByte`). If the address has no chip, that is a fault.
- The new `trace.enqueue(...)` line records the instruction in the flight recorder (Part 2.3).
- **Decode:** `OpcodeTable::lookup(opcode)` is an array index (Part 2.1). If the slot is marked invalid, the CPU stops with `InvalidOpcode`.
- **Execute:** first read the operand bytes (their layout depends only on the format), check the register numbers, then a `switch(opcode)` does the work.

This is how `execute()` reads operands. Look at **RegReg**: one byte holds two register numbers, split with a shift and a mask:

CPU/Aashish/src/CPU.cpp · lines 90–115

```
90      switch(info.format){
91          case Format::Implied:
92              break;
93          case Format::Reg:
94          case Format::RegInd:
95              a = fetchByte();
96              break;
97          case Format::RegReg:{
98              Byte both = fetchByte();
99              a = (Byte)(both >> 4);      //destination in the high 4 bits
100             b = (Byte)(both & 0x0F);    //source in the low 4 bits
101             break;
102         }
103         case Format::RegImm:
104             a = fetchByte();
105             b = fetchByte();
106             break;
107         case Format::Abs:
108             addr = fetchWord();
109             break;
110         case Format::RegAbs:
111             a = fetchByte();
112             addr = fetchWord();
113             break;
114     }
115     if(busFailed()) return;
```

1.7 Worked example: 5 + 3 on the real CPU

The program below was loaded into the real `Console` and stepped one instruction at a time. After each step it printed `std::cout << cpu` (the friend function from Part 3.7). **This is real output, not made up.**

Address	Bytes	Assembly	Meaning
`0x0000`	`11 00 05`	`LDI R0, 5`	R0 = 5
`0x0003`	`11 01 03`	`LDI R1, 3`	R1 = 3
`0x0006`	`30 01`	`ADD R0, R1`	R0 = R0 + R1
`0x0008`	`01`	`HLT`	stop

output of the program, run on the CPU (Console + Disassembler + operator<<)

```
start : PC=0x0000 SP=0xFFFF R0=0x00 R1=0x00 R2=0x00 R3=0x00 R4=0x00 R5=0x00 R6=0x00 R7=0x00 flags=---- Running cycles=0
0x0000  LDI R0, 0x05
   -> PC=0x0003 SP=0xFFFF R0=0x05 R1=0x00 R2=0x00 R3=0x00 R4=0x00 R5=0x00 R6=0x00 R7=0x00 flags=---- Running cycles=3
0x0003  LDI R1, 0x03
   -> PC=0x0006 SP=0xFFFF R0=0x05 R1=0x03 R2=0x00 R3=0x00 R4=0x00 R5=0x00 R6=0x00 R7=0x00 flags=---- Running cycles=6
0x0006  ADD R0, R1
   -> PC=0x0008 SP=0xFFFF R0=0x08 R1=0x03 R2=0x00 R3=0x00 R4=0x00 R5=0x00 R6=0x00 R7=0x00 flags=---- Running cycles=8
0x0008  HLT
   -> PC=0x0009 SP=0xFFFF R0=0x08 R1=0x03 R2=0x00 R3=0x00 R4=0x00 R5=0x00 R6=0x00 R7=0x00 flags=---- Halted cycles=9
trace size 4: 0 3 6 8
```

- PC jumps by the **length** of each instruction: 3, 3, 2, 1 bytes.
- `cycles` adds 3 + 3 + 2 + 1 = 9. An instruction's cycle cost is the bytes it fetches plus any extra memory reads/writes.
- 8 is not zero, has no carry and bit 7 is 0, so all flags stay off (`----`).
- The trace recorded the four start addresses 0, 3, 6, 8, oldest first.

1.8 The ALU: how flags are set

ADD, ADC and ADDI all share one function. It works out the true sum in a bigger `unsigned int`, keeps the low 8 bits, and sets the flags:

CPU/Aashish/src/CPU.cpp · lines 286–294

```
286  //shared by ADD, ADC, ADDI: C = true sum did not fit in 8 bits,
287  //V = both inputs had the same sign but the result has the other sign
288  Byte CPU::add(Byte a, Byte b, bool carryIn){
289      unsigned int sum = (unsigned int)a + b + (carryIn ? 1 : 0);
290      Byte result = (Byte)(sum & 0xFF);
291      setZN(result);
292      setFlag(FLAG_C, sum > 0xFF);
293      setFlag(FLAG_V, ((~(a ^ b)) & (a ^ result) & 0x80) != 0);
294      return result;
```

Example	8-bit result	Z	C	N	V	Why
200 + 100	`0x2C` (44)	0	1	0	0	300 does not fit in 8 bits: C = 1, result wraps to 300 − 256
100 + 50	`0x96` (150)	0	0	1	1	fits unsigned, but as signed numbers two positives gave a negative: V = 1
5 + 3	`0x08`	0	0	0	0	an ordinary sum

C is for unsigned numbers (0–255), **V** for signed numbers (−128 to 127). The V formula reads: "both inputs had the same sign, and the result's sign is different".

1.9 Faults: a bad program stops cleanly

A broken program must never crash the emulator. Instead the CPU records a `Fault`, sets its state to `Faulted` and stops. There are 7 kinds:

Fault	Caused by
`InvalidOpcode`	a byte that is not an instruction, e.g. `0xFF`
`InvalidRegister`	register number above 7
`IllegalWrite`	writing into ROM, or into a read-only port
`UnmappedAccess`	an address no chip owns (possible on test boards that leave gaps)
`StackOverflow` / `StackUnderflow`	PUSH when the stack is full / POP when it is empty
`InvalidJumpTarget`	JMP, CALL or RET to an address outside ROM

CPU/Aashish/src/CPU.cpp · lines 358–362

```
358  //PC is moved back to the failing instruction, so the debugger shows the line that caused it
359  void CPU::raise(FaultType type, Word address, const std::string& message){
360      fault.type = type;
361      fault.pc = instrStart;
362      fault.address = address;
```

PC is moved back to where the failing instruction **started**, so a debugger highlights the line that caused the problem, not the one after it. Expected errors are returned as data (`bool`, `StepResult`, `Fault`), not thrown as exceptions.

1.10 The Console: the whole machine in one object

Other parts of the project (Arnav's assembler, Ashmit's screen, Piyush's State Manager) use the CPU through one class. Its constructor builds every chip and plugs each into the Bus at its address range from the memory map:

CPU/Aashish/src/Console.cpp · lines 12–25

```
12  Console::Console()
13      : romChip(sizeOf(MemoryMap::ROM_START, MemoryMap::ROM_END)),
14        ramChip(sizeOf(MemoryMap::RAM_START, MemoryMap::RAM_END), "RAM"),
15        vramChip(sizeOf(MemoryMap::VRAM_START, MemoryMap::VRAM_END), "VRAM"),
16        ioChip(),
17        stackChip(sizeOf(MemoryMap::STACK_START, MemoryMap::STACK_END), "STACK"),
18        theBus(),
19        theCpu(theBus){
20      theBus.attach(MemoryMap::ROM_START,   MemoryMap::ROM_END,   &romChip);
21      theBus.attach(MemoryMap::RAM_START,   MemoryMap::RAM_END,   &ramChip);
22      theBus.attach(MemoryMap::VRAM_START,  MemoryMap::VRAM_END,  &vramChip);
23      theBus.attach(MemoryMap::IO_START,    MemoryMap::IO_END,    &ioChip);
24      theBus.attach(MemoryMap::STACK_START, MemoryMap::STACK_END, &stackChip);
25  }
```

A game runs one **frame** at a time: the CPU gets a budget of cycles, then the frame counter ticks so the program can wait for the next frame (like vsync on a real console):

CPU/Aashish/src/Console.cpp · lines 52–63

```
52  StepResult Console::runFrame(unsigned long cycleBudget){
53      unsigned long stopAt = theCpu.getCycles() + cycleBudget;
54      StepResult result = StepResult::Ok;
55
56      while(theCpu.getCycles() < stopAt){
57          result = theCpu.step();
58          if(result != StepResult::Ok) break;
59      }
60
61      ioChip.nextFrame();
62      return result;
63  }
```

Check yourself

- What does `LDI R2, 0x10` look like in bytes? (Answer: `11 02 10`.)
- After `CALL`, why is the return address already in PC without any extra work? (Hint: when is PC incremented?)
- Which chip answers address `0xE020`, and which pixel is it? (VRAM, x = 0, y = 1.)

Part 2 · Data structures in the CPU

The DS syllabus topics, where each one lives, and why it was the right choice there.

Syllabus topic	Where	Status
Array	register file, 256-slot opcode table, memory chips, queue storage	used from the start
Stack	`CPU::push`/`pop`, `CALL`/`RET`, `PUSHF`/`POPF`; step-back history in `live.cpp`	used from the start
Queue (circular)	`InstructionTrace`, filled on every `CPU::step`	added
Binary search	`Bus::find` over the sorted chip ranges	added (replaced a linear search)
Hash map	`OpcodeTable::find`: name → opcode, used by the assembler	added (replaced a linear search)
Linked list	not used, on purpose (2.6 explains why)	deliberately not used

2.1 Array

> **The idea in one line**
> A fixed block of same-type items side by side in memory. Item `i` is found by arithmetic (`start + i × size`), so access by index is **O(1)**: no searching.

Where

- Register file: `Byte reg[NUM_REGS]` in `CPU.h:55`, so register `R3` is `reg[3]`.
- Opcode table: `OpcodeInfo slot[256]` in `Opcodes.cpp:66`. The opcode byte **is** the index, so decoding never searches.
- Memory chips: `std::vector<Byte> data` in `Ram.h:21`, a dynamic array. The address offset is the index.
- The queue's storage: `TraceEntry items[CAPACITY]` in `InstructionTrace.h:32`.

CPU/Aashish/src/Opcodes.cpp · lines 64–84

```
64      //256 slots, one per possible byte, so lookup is a plain array index: O(1)
65      struct Table{
66          OpcodeInfo slot[256];
67
68          Table(){
69              OpcodeInfo invalid = {"???", Format::Implied, 1, 0, false};
70              for(int i = 0; i < 256; ++i) slot[i] = invalid;
71
72              for(int i = 0; i < ENTRY_COUNT; ++i){
73                  const Entry& e = ENTRIES[i];
74                  OpcodeInfo info = {e.mnemonic, e.format, OpcodeTable::lengthOf(e.format), e.cycles,
                        true};
75                  slot[e.opcode] = info;
76              }
77          }
78      };
79
80      //built the first time it is needed (safe even if another file's globals call lookup early)
81      const Table& table(){
82          static const Table t;
83          return t;
84      }
```

CPU/Aashish/src/Ram.cpp · lines 7–16

```
7   Byte Ram::read(Word offset){
8       if(offset >= data.size()) return 0xFF;
9       return data[offset];
10  }
11
12  bool Ram::write(Word offset, Byte value){
13      if(offset >= data.size()) return false;
14      data[offset] = value;
15      return true;
16  }
```

Why this structure here?

The number of registers (8) and of possible opcodes (256) is fixed, and both are always reached by a number. That is exactly what arrays are best at. Every unused opcode slot is marked `valid = false`, so a bad byte is caught in one step.

Say this in the viva

"The registers are an array of 8 bytes, so R3 is just `reg[3]`. To decode an instruction I index a 256-entry array with the opcode byte itself, so decoding is O(1) with no searching. Memory chips are `std::vector`s, a dynamic array, and the address offset is the index."

2.2 Stack

The idea in one line

LIFO: last in, first out. `push` puts an item on top, `pop` takes the top one off. Pushing onto a full stack is **overflow**; popping an empty one is **underflow**.

This is the question I could not answer in the evaluation, so here it is clearly. The CPU has a real **array-based stack**:

Stack idea	In my CPU
the array	the `STACK` chip: a `Ram` of 3,840 bytes at `0xF100`–`0xFFFF` (a `std::vector<Byte>`)
top index	`SP`, the stack pointer register
push	write the value at `SP`, then `SP--` (it grows **down**)
pop	`SP++`, then read the value at `SP`
overflow check	`SP < 0xF100`: no room left, raise `StackOverflow`
underflow check	`SP == 0xFFFF`: nothing on it, raise `StackUnderflow`

CPU/Aashish/src/CPU.cpp · lines 308–328

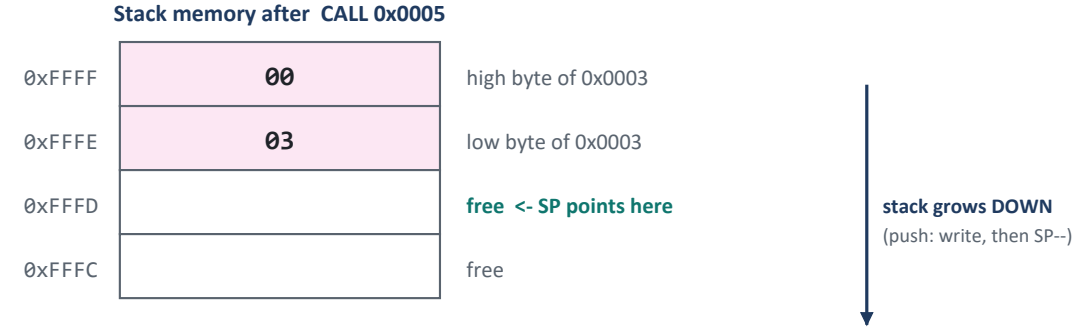
```
308  //stack grows DOWN; SP points at the next free slot
309  bool CPU::push(Byte value){
310      if(sp < MemoryMap::STACK_START){
311          raise(FaultType::StackOverflow, sp, "Stack is full");
312          return false;
313      }
314      bus.write(sp, value);
315      if(busFailed()) return false;
316      sp = (Word)(sp - 1);
317      return true;
318  }
319
320  bool CPU::pop(Byte& value){
321      if(sp == MemoryMap::STACK_END){
322          raise(FaultType::StackUnderflow, sp, "Stack is empty");
323          return false;
324      }
325      sp = (Word)(sp + 1);
326      value = bus.read(sp);
327      return !busFailed();
328  }
```

Who uses it

CALL and RET. `CALL` pushes the return address (high byte, then low byte) and jumps. `RET` pops it back off. Because the stack is LIFO, nested calls return in the right order: if A calls B and B calls C, C returns to B before B returns to A.

CPU/Aashish/src/CPU.cpp · lines 238–252

```
238          case OP_CALL:
239              if(!checkJump(addr)) return;
240              //pc already points at the next instruction: that is the return address
241              if(!push(highByte(pc))) return;
242              if(!push(lowByte(pc))) return;
243              pc = addr;
244              break;
245          case OP_RET:{
246              Byte low = 0, high = 0;
247              if(!pop(low)) return;
248              if(!pop(high)) return;
249              Word target = makeWord(low, high);
250              if(checkJump(target)) pc = target;
251              break;
252          }
```



Real values, read with bus.peek() right after the CALL instruction ran.

The real stack after running `CALL 0x0005` from address `0x0000` (real output: `SP=fffd [FFFF]=0 [FFFE]=3 PC=5`; after the `RET`: `SP=ffff PC=3`).

PUSH / POP / PUSHF / POPF let a program save a register or the flags and get them back later (`CPU.cpp:255–270`).

Undo in the live demo is a second stack. Every step is pushed onto `hist` (`hist.push_back(r)`, `live.cpp:463`). Step-back pops the newest one and undoes its memory writes newest first:

CPU/Aashish/demo/live.cpp · lines 468–483

```
468         void back(){
469             if(hist.empty()) return;
470             const StepRec& r = hist.back();
471             for(size_t i = r.writes.size(); i-- > 0; ){
472                 board.bus.poke(r.writes[i].address, r.writes[i].oldValue);
473                 owner[r.writes[i].address] = r.writes[i].prevOwner;
474             }
475             for(int i = 0; i < CPU::NUM_REGS; ++i) board.cpu.setReg(i, r.before.reg[i]);
476             board.cpu.setPC(r.before.pc);
477             board.cpu.setSP(r.before.sp);
478             board.cpu.setFlags(r.before.flags);
479             board.cpu.setCycles(r.before.cycles);
480             board.cpu.setState(CpuState::Running);      //it was running, or it could not have stepped
481             board.bus.clearFault();
482             hist.pop_back();
483         }
```

Why this structure here?

A return address must come back in the **reverse** order it went in, and so must undo history. Reverse order is exactly what LIFO gives. It sits at the top of memory and grows down, so it never collides with the program or the variables.

Say this in the viva

"My CPU has an array-based stack. The array is the stack memory chip from F100 to FFFF, and SP is the top index. Push writes at SP and decrements it; pop increments SP and reads. I check overflow when SP goes below F100 and underflow when SP is FFFF. CALL pushes the return address and RET pops it, so nested function calls return in the right order. That's in `CPU::push` and `CPU::pop` in CPU.cpp."

Likely question

Why does the stack grow downward? It starts at the top of memory, so all the free space stays in one piece. Most real CPUs do the same. **What happens on overflow?** `push` refuses and the CPU stops with `StackOverflow`, PC pointing at the PUSH. Nothing is overwritten.

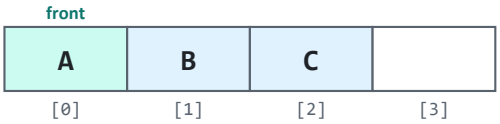
2.3 Queue (circular)

The idea in one line

FIFO: first in, first out. Add at the **rear** (`enqueue`), remove from the **front** (`dequeue`). A **circular** queue keeps a fixed array and lets `front` and `rear` wrap around the end with `%`, so nothing ever has to be shifted.

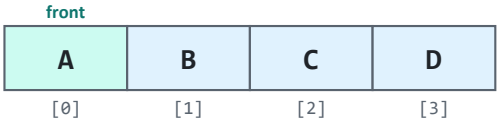
What I use it for: an instruction trace, like an aircraft's flight recorder. The CPU enqueues every instruction it starts. When a program crashes, the trace shows the last 16 instructions, oldest first, so you can see **how** the program got to the bad line, not just where it stopped.

enqueue A, B, C



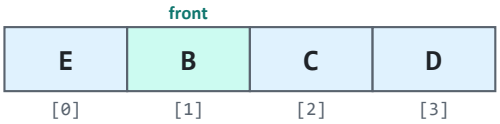
front = 0, count = 3, rear = (0+3) % 4 = 3

enqueue D (now full)



front = 0, count = 4, isFull() is true

enqueue E (full, so A is dequeued first)



front = 1, E written at (1+3) % 4 = 0 -> oldest to newest: B C D E

Teaching example with CAPACITY = 4. The real InstructionTrace uses CAPACITY = 16, same code.

How the circular queue behaves when it fills up. The oldest entry is dropped, the new one reuses its slot, and `front` moves on.

CPU/Aashish/src/InstructionTrace.h · lines 13–35

```
13  //a circular queue in a fixed array. front is the oldest entry, the rear is front + count,
14  //and both wrap around the end of the array. When it is full, enqueue drops the oldest entry
15  //first, so it always holds the LAST CAPACITY instructions and never grows, however long the program
        runs.
16  class InstructionTrace{
17      public:
18          static const int CAPACITY = 16;
19
20          InstructionTrace();
21
22          void enqueue(Word pc, Byte opcode);     //adds at the rear
23          bool dequeue(TraceEntry& entry);        //takes the oldest off the front; false if empty
24          void clear();
25
26          bool isEmpty() const;
27          bool isFull() const;
28          int  size() const;
29          TraceEntry at(int i) const;             //0 = oldest, size()-1 = newest; {0, 0} outside that
30
31      private:
32          TraceEntry items[CAPACITY];
33          int front;
34          int count;
35  };
```

CPU/Aashish/src/InstructionTrace.cpp · lines 9–26

```
 9  void InstructionTrace::enqueue(Word pc, Byte opcode){
10      if(isFull()){
11          TraceEntry oldest;
12          dequeue(oldest);
13      }
14      int rear = (front + count) % CAPACITY;
15      items[rear].pc = pc;
16      items[rear].opcode = opcode;
17      count++;
18  }
19
20  bool InstructionTrace::dequeue(TraceEntry& entry){
21      if(isEmpty()) return false;
22      entry = items[front];
23      front = (front + 1) % CAPACITY;
24      count--;
25      return true;
26  }
```

CPU/Aashish/src/InstructionTrace.cpp · lines 49–55

```
49  TraceEntry InstructionTrace::at(int i) const{
50      if(i < 0 || i >= count){
51          TraceEntry none = {0, 0};
52          return none;
53      }
54      return items[(front + i) % CAPACITY];
55  }
```

The two formulas to remember: the **rear** (next free slot) is `(front + count) % CAPACITY`, and the `i`-th oldest item is `items[(front + i) % CAPACITY]`. The CPU feeds it in `step()`:

CPU/Aashish/src/CPU.cpp · lines 45–47

```
45      Byte opcode = fetchByte();
46      if(busFailed()) return StepResult::Fault;
47      trace.enqueue(instrStart, opcode);      //before decoding, so a bad opcode is recorded too
```

And `demo.cpp` (Part 5 of the demo) prints it after a crash. Real output, after a program that pushes forever:

demo.exe, Part 5: fault handling

```
The last 4 of the 16 instructions the CPU keeps in its trace (newest = the one that failed):
  0x0002  JMP 0x0000
  0x0000  PUSH R0
  0x0002  JMP 0x0000
  0x0000  PUSH R0
```

Why this structure here?

- I want only the **last** N instructions, and the program may run for millions of steps. A fixed array means fixed memory, however long it runs.
- Enqueue and dequeue are **O(1)**: no shifting, unlike a simple array queue where removing from the front moves everything down.
- `std::queue` doesn't fit: it has no fixed capacity, and you cannot read its middle items to print them.

Say this in the viva

"I added an instruction trace as a circular queue of 16 entries in a fixed array. Every step, the CPU enqueues the PC and opcode at the rear, `(front + count) % 16`. When it's full I dequeue the oldest first, so it always holds the last 16 instructions in O(1) with fixed memory. After a crash it shows how the program got there."

Likely question

Why circular and not a normal array queue? A normal one either shifts every item on dequeue, O(n), or runs off the end of the array. Circular reuses the freed slots by wrapping the index with `%`.

2.4 Binary search

The idea in one line

On **sorted** data: look at the middle item; if the target is smaller, throw away the right half, if larger, the left half. Repeat. Each step halves the search, so it takes **O(log n)** steps instead of O(n).

Where: the Bus has to find which chip owns an address, on **every** memory access the CPU makes. The chip ranges never overlap, so if they are kept sorted by start address, binary search works. `attach()` keeps them sorted by inserting each new range in order:

CPU/Aashish/src/Bus.cpp · lines 30–33

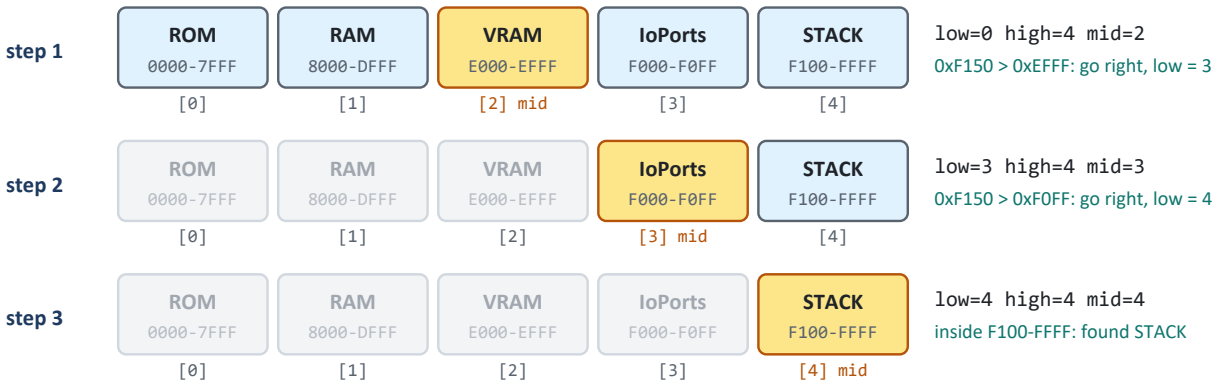
```
30      //insert it in order of start address, so the list stays sorted for find()
31      size_t pos = 0;
32      while(pos < mappings.size() && mappings[pos].start < start) ++pos;
33      mappings.insert(mappings.begin() + pos, m);
```

CPU/Aashish/src/Bus.cpp · lines 37–51

```
37  //binary search: the ranges are sorted and never overlap, so look at the middle one and throw
38  //away the half the address cannot be in. O(log n) checks instead of trying every chip.
39  //This runs on every memory access the CPU makes, and attach() only runs at start-up.
40  Bus::Mapping* Bus::find(Word address){
41      int low = 0;
42      int high = (int)mappings.size() - 1;
43
44      while(low <= high){
45          int mid = low + (high - low) / 2;
46          if(address < mappings[mid].start)    high = mid - 1;        //it is in the left half
47          else if(address > mappings[mid].end) low = mid + 1;         //it is in the right half
48          else return &mappings[mid];
49      }
50      return nullptr;         //it fell in a gap between chips
51  }
```

It is binary search on **ranges** instead of single values, so each step has three outcomes: the address is left of the middle range, right of it, or inside it (found). Here is a real lookup:

find(0xF150) on the real Console's 5 mappings (sorted by start address)



Yellow = the middle range checked; grey = the half already thrown away. 3 steps, not 5.

Looking up `0xF150` (a stack address) in the real machine's five chips.

Why this structure here?

The ranges are sorted and non-overlapping, which is exactly when binary search works, and the lookup runs on every single memory access. The sorting cost is paid once, in `attach()`, which only runs at start-up.

Careful

Be honest about the size: with only 5 chips, binary search needs at most 3 steps instead of 5, a small difference. The point is that the method is the right one for sorted, non-overlapping ranges, and it stays O(log n) if more devices are added. `mid` is written `low + (high - low) / 2` instead of `(low + high) / 2`, the safe form that cannot overflow.

Say this in the viva

"The Bus keeps its chip ranges sorted by start address. `attach` inserts each range in order, and `find` does a binary search: compare the address with the middle range, go left or right, or return it if the address is inside. It's O(log n) and runs on every memory access."

Likely question

What if two ranges overlapped? Then one address could match two chips and the search would be ambiguous. That is why `attach()` refuses any overlapping range. A test checks this.

2.5 Hash map

The idea in one line

Stores **key → value** pairs. A **hash function** turns the key into a bucket number, so lookup jumps straight to the right bucket: **O(1) on average**. Two keys landing in the same bucket is a **collision**; `std::unordered_map` keeps both in that bucket's list.

The CPU needs lookups in two directions, and each uses the best tool:

Direction	Who needs it	Structure	Cost
opcode byte → instruction info	the CPU, on every instruction	256-slot array, the byte is the index (a **direct-address table**, the simplest kind of hash table)	O(1)
name ("ADD") → opcode byte	the assembler and operand checker, on every line of a program	`std::unordered_map<std::string, Byte>`	O(1) average

CPU/Aashish/src/Opcodes.cpp · lines 86–99

```
86      //mnemonic -> opcode, a hash map: the name is hashed to a bucket, so a lookup is O(1) on average
87      //instead of comparing against all 38 names. The assembler and the operand checker look up every
            line.
88      struct Names{
89          std::unordered_map<std::string, Byte> opcodeOf;
90
91          Names(){
92              for(int i = 0; i < ENTRY_COUNT; ++i) opcodeOf[ENTRIES[i].mnemonic] = ENTRIES[i].opcode;
93          }
94      };
95
96      const Names& names(){
97          static const Names n;
98          return n;
99      }
```

CPU/Aashish/src/Opcodes.cpp · lines 106–115

```
106 //case-insensitive, so "add", "Add" and "ADD" from the parser all work: the keys are upper case,
107 //so the name is made upper case before it is hashed
108 int OpcodeTable::find(const std::string& mnemonic){
109     std::string key = mnemonic;
110     for(size_t i = 0; i < key.size(); ++i) key[i] = (char)std::toupper((unsigned char)key[i]);
111
112     std::unordered_map<std::string, Byte>::const_iterator it = names().opcodeOf.find(key);
113     if(it == names().opcodeOf.end()) return -1;
114     return it->second;
115 }
```

The keys are stored in upper case, and the name is changed to upper case before it is hashed, so `add`, `Add` and `ADD` all find `0x30`. Arnav's assembler calls it for every instruction it encodes:

CPU/src/Assembler.cpp · line 105 · Arnav's assembler

```
105         Byte opcode = static_cast<Byte>(OpcodeTable::find(instruction.name));
```

Say this in the viva

"Opcode to instruction is a direct-address table: the opcode byte is the array index. Name to opcode, which the assembler needs for every line, is an `unordered_map`, a hash map, so the lookup is O(1) on average instead of comparing the name against all 38 instructions."

Likely question

Why not a hash map for opcodes too? An opcode is already a small number, 0–255, so it can be the index itself. No hashing needed: a direct-address table is the perfect hash.

2.6 Linked list: deliberately not used

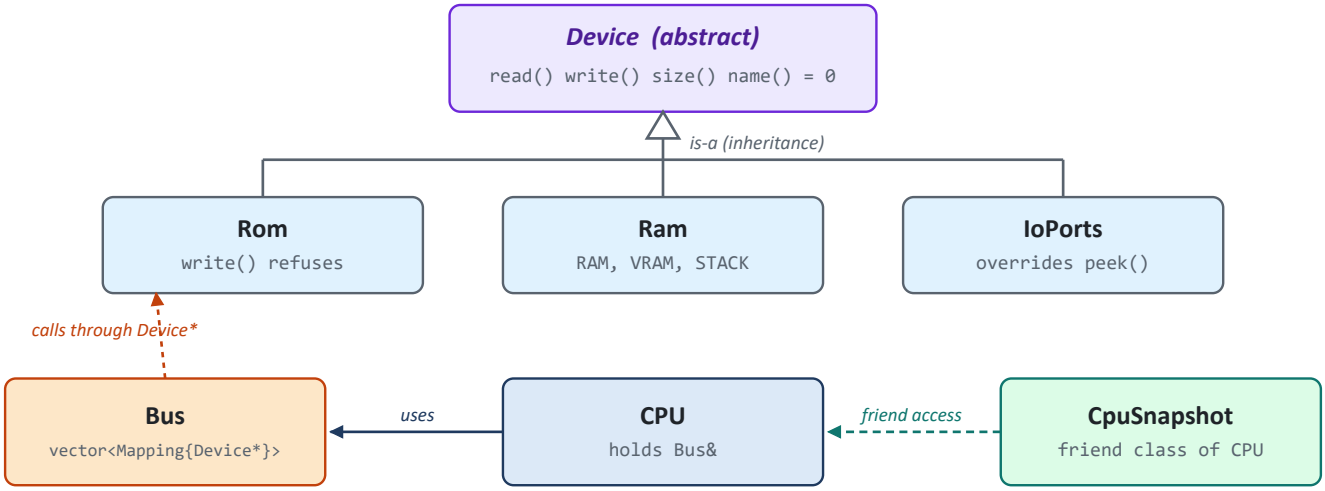
> **The idea in one line**
> Nodes scattered in memory, each pointing to the next. Inserting or removing in the **middle** is O(1) if you already have the node, but finding item `i` means walking from the start: **O(n)**, and there is no binary search.

Every list in the CPU is either **fixed-size** (registers, opcode table, trace) or **read by position** (memory by address, the sorted chip list that binary search needs). Nothing inserts or removes in the middle. A linked list would make all of these slower and use extra memory for the pointers.

> **Say this in the viva**
> "I didn't use a linked list because nothing in the CPU fits one. My lists are fixed-size or accessed by index, and the Bus list needs binary search, which needs random access. A linked list is the right choice when you insert and delete in the middle a lot, and the CPU never does. In our project it would fit undo/redo history in the State Manager: add and drop at both ends in O(1) with a cursor that moves back and forth."

Part 3 · C++ and OOP in the CPU

Classes and objects, the four pillars, constructors, friends and static members, each with the file and line numbers.



How the main classes relate. Solid lines with a hollow triangle are inheritance (is-a); the arrows are uses / has-a.

3.1 Classes and objects

A **class** is a blueprint: the data and the functions that work on it. An **object** is one real thing built from it. The clearest example in the project: there is **one** `Ram` class, and the Console makes **three** `Ram` objects from it, each with its own size and name.

CPU/Aashish/src/Console.h · lines 44–52

```
44          //order matters: members are built top to bottom, and the CPU needs the Bus,
45          //which needs the chips to exist first
46          Rom     romChip;
47          Ram     ramChip;
48          Ram     vramChip;
49          IoPorts ioChip;
50          Ram     stackChip;
51          Bus     theBus;
52          CPU     theCpu;
```

The `Console` constructor in Part 1.10 gives each one its size: `ramChip(..., "RAM")`, `vramChip(..., "VRAM")`, `stackChip(..., "STACK")`. Same class, three objects, three different chips.

> **Say this in the viva**
> "A class is the blueprint and an object is an instance. I wrote one `Ram` class and the Console creates three objects from it: RAM, VRAM and the stack, each with a different size."

3.2 Encapsulation

> **The idea in one line**
> Keep the data `private` and let the outside world change it only through `public` functions that check what they're given.

Everything inside the CPU (registers, PC, SP, flags, state) is private. Other code goes through methods, and the methods guard the data. `setReg` ignores a register number that does not exist, instead of writing past the end of the array:

CPU/Aashish/src/CPU.cpp · lines 373–376

```
373  void CPU::setReg(int index, Byte value){
374      if(index < 0 || index >= NUM_REGS) return;
375      reg[index] = value;
376  }
```

CPU/Aashish/src/CPU.h · lines 33–43

```
33          //view for the debugger and tests
34          Byte  getReg(int index) const;
35          void  setReg(int index, Byte value);
36          Word  getPC() const;
37          Word  getSP() const;
38          Word  getP() const;                         //R6:R7 as one 16-bit pointer
39          Byte  getFlags() const;
40          bool  getFlag(Flag flag) const;
41          CpuState getState() const;
42          const Fault& getFault() const;
43          unsigned long getCycles() const;
```

Other examples: `setFlags` keeps only bits 0–3; `Rom` has no public way to change bytes except `load()`; the Bus's list of chips is private, so nobody can break its sorted order.

> **Say this in the viva**
> "All CPU state is private. Getters are `const` so they can't change anything, and setters validate their input. For example `setReg` refuses register numbers above 7, so nothing can write outside the register array."

3.3 Abstraction

> **The idea in one line**
> Show **what** something does and hide **how**. In C++: an abstract class with **pure virtual** functions (`= 0`). It can't be instantiated; it only defines the interface.

CPU/Aashish/src/Device.h · lines 8–20

```
8   class Device{
9       public:
10          virtual ~Device() {}
11
12          virtual Byte read(Word offset) = 0;
13          virtual bool write(Word offset, Byte value) = 0;   //false = refused
14          virtual unsigned int size() const = 0;
15          virtual const char* name() const = 0;
16
17          //read without side effects, for debuggers and undo. Plain memory just reads;
18          //IoPorts overrides it so looking at the RNG port does not use up a random number.
19          virtual Byte peek(Word offset) { return read(offset); }
20  };
```

The Bus only knows this interface: "a chip can read, write, tell its size and its name". It has no idea whether it is talking to ROM, RAM or the I/O ports, and it doesn't need to.

Say this in the viva

"`Device` is an abstract class with pure virtual `read`, `write`, `size` and `name`. The Bus only depends on that interface, so I can add a new chip, a sound chip for example, without changing the Bus at all."

3.4 Inheritance

The idea in one line

A derived class **is-a** base class: it gets the base's interface and fills in or changes the behaviour. `override` makes the compiler check that the signature really matches.

CPU/Aashish/src/Ram.h · lines 9–23

```
 9  class Ram : public Device{
10      public:
11          Ram(unsigned int size, const char* name);
12
13          Byte read(Word offset) override;
14          bool write(Word offset, Byte value) override;
15          unsigned int size() const override;
16          const char* name() const override;
17
18          void clear();       //set every byte to 0
19
20      private:
21          std::vector<Byte> data;
22          const char* label;
23  };
```

`Rom` and `IoPorts` inherit from `Device` the same way. Each fills in the interface differently. ROM refuses every write, which is what makes it read-only:

CPU/Aashish/src/Rom.cpp · lines 11–14

```
11  //the CPU can never change ROM; the Bus turns this "false" into an IllegalWrite fault
12  bool Rom::write(Word, Byte){
13      return false;
14  }
```

Say this in the viva

"`Rom`, `Ram` and `IoPorts` all inherit publicly from `Device`. Each one overrides `read` and `write` its own way: RAM stores the byte, ROM refuses, and the I/O ports react, for example reading the RNG port gives the next random number."

3.5 Polymorphism

Run-time polymorphism (virtual functions)

The Bus stores `Device*` pointers and calls `device->read()`. Which `read` actually runs is decided **at run time** by the real type of the object: `Ram::read`, `Rom::read` or `IoPorts::read`.

CPU/Aashish/src/Bus.cpp · lines 53–60

```
53  Byte Bus::read(Word address){
54      Mapping* m = find(address);
55      if(m == nullptr){
56          raise(FaultType::UnmappedAccess, address, "No device at " + hex16(address));
57          return 0xFF;
58      }
59      return m->device->read((Word)(address - m->start));
60  }
```

`Device` also gives `peek` a default body (just call `read`). `IoPorts` **overrides** it, because reading the RNG port uses up a random number and a debugger must be able to look without changing anything:

CPU/Aashish/src/IoPorts.cpp · lines 38–45

```
38  //only the RNG port changes when it is read. 8 steps = 8 fresh bits, so a number is not
39  //just the last one shifted by one. (8 and 65535 share no factor, so all states are still visited.)
40  Byte IoPorts::read(Word offset){
41      if(offset == OFF_RNG){
42          for(int i = 0; i < 8; ++i) stepRandom();
43      }
44      return peek(offset);
45  }
```

Compile-time polymorphism (overloading)

- **Function overloading:** two `loadProgram` functions, one takes a raw array and one a `std::vector` (`Console.h:26–27`).
- **Constructor overloading:** `CpuSnapshot()` and `CpuSnapshot(const CPU&)` (Part 3.6).
- **Operator overloading:** `operator<<` for the CPU (Part 3.7).

> **Say this in the viva**
> "Run-time polymorphism: the Bus calls `device->read()` through a base-class pointer, and the virtual function picks Ram, Rom or IoPorts at run time. Compile-time polymorphism: `loadProgram` is overloaded for arrays and vectors, and I overloaded `operator<<` for the CPU."

3.6 Constructors

Kind	Where	What it shows
Constructor with an initializer list	`Console::Console()`, `Console.cpp:12–19`	members are built in the list, before the body runs
`explicit` single-argument constructor	`explicit CPU(Bus& bus)`, `CPU.h:27`	stops a `Bus` being silently converted into a `CPU`
Default constructor	`Fault()`, `Fault.h:24`	a new Fault always means "no error"
Constructor overloading	`CpuSnapshot()` and `explicit CpuSnapshot(const CPU&)`	an empty save point, or one copied from a CPU
Parameterised constructor	`Ram(unsigned int size, const char* name)`, `Ram.cpp:4`	the same class makes different-sized chips
Virtual destructor	`virtual ~Device() {}`, `Device.h:10`	deleting through a `Device*` runs the right destructor

CPU/Aashish/src/CPU.cpp · lines 20–22

```
20  CPU::CPU(Bus& theBus) : bus(theBus){
21      reset(0x0000);
22  }
```

The CPU holds a **reference** to the Bus (`Bus& bus`). A reference must be bound when the object is created, so it **has** to be set in the initializer list, not in the body. A reference can never be null, so a CPU is always connected to a Bus.

CPU/Aashish/src/CpuSnapshot.cpp · lines 4–15

```
4   //the same values CPU::reset(0x0000) sets
5   CpuSnapshot::CpuSnapshot()
6       : pc(0x0000), sp(MemoryMap::STACK_END), flags(0), state(CpuState::Running),
7         fault(), cycles(0), instrStart(0x0000), trace(){
8       for(int i = 0; i < CPU::NUM_REGS; ++i) reg[i] = 0;
9   }
10
11  CpuSnapshot::CpuSnapshot(const CPU& cpu)
12      : pc(cpu.pc), sp(cpu.sp), flags(cpu.flags), state(cpu.state),
13        fault(cpu.fault), cycles(cpu.cycles), instrStart(cpu.instrStart), trace(cpu.trace){
14      for(int i = 0; i < CPU::NUM_REGS; ++i) reg[i] = cpu.reg[i];
15  }
```

Careful

Member order matters. Members are built in the order they are **declared** in the class, not the order of the initializer list. The CPU needs the Bus, and the Bus needs the chips, so `Console.h` declares the chips first and the CPU last (see the comment `//order matters` in the code box in 3.1).

Say this in the viva

"I use initializer lists everywhere. The CPU takes the Bus by reference, which must be initialised in the list. Its constructor is `explicit`, `Device` has a virtual destructor, and `CpuSnapshot` has two overloaded constructors: an empty save point and one copied from a CPU."

3.7 Friend function: operator<<

The idea in one line

A **friend function** is not a member of the class, but the class lets it access the private members. The classic use is `operator<<`.

CPU/Aashish/src/CPU.h · lines 16–21

```
16  class CPU{
17      //a save point copies the private parts directly, the fault included, which no public setter can put
            back
18      friend class CpuSnapshot;
19
20      //std::cout << cpu: a register dump on one line. Not a member, because the stream is on the left of
            <<
21      friend std::ostream& operator<<(std::ostream& out, const CPU& cpu);
```

CPU/Aashish/src/CPU.cpp · lines 437–453

```
437 //e.g. "PC=0x0007 SP=0xFFFF R0=0x2A ... R7=0x80 flags=--N- Halted cycles=7"
438 //a friend, so it reads the private registers directly instead of going through the getters
439 std::ostream& operator<<(std::ostream& out, const CPU& cpu){
440     out << "PC=" << hex16(cpu.pc) << " SP=" << hex16(cpu.sp);
441     for(int i = 0; i < CPU::NUM_REGS; ++i) out << " R" << i << "=" << hex8(cpu.reg[i]);
442
443     const char names[] = "ZCNV";                //same order as the Flag bit positions
444     out << " flags=";
445     for(int i = 0; i < 4; ++i) out << (getBit(cpu.flags, i) ? names[i] : '-');
446
447     if(cpu.state == CpuState::Running)     out << " Running";
448     else if(cpu.state == CpuState::Halted) out << " Halted";
449     else                                   out << " Faulted (" << faultName(cpu.fault.type) << ")";
450
451     out << " cycles=" << cpu.cycles;
452     return out;
453 }
```

Why it can't be a member function: in `std::cout << cpu`, the object on the **left** of `<<` is the stream, not the CPU. A member function always has its own class on the left. So it must be a non-member, and making it a friend lets it read `pc`, `sp`, `reg[]` and `flags` directly. It returns the stream so calls can be chained: `cout << "[" << cpu << "]"`.

std::cout << cpu (real output, from the demo and the 5 + 3 program)

```
PC=0x0009 SP=0xFFFF R0=0x08 R1=0x03 R2=0x00 R3=0x00 R4=0x00 R5=0x00 R6=0x00 R7=0x00 flags=---- Halted cycles=9
PC=0x0000 SP=0xF0FF R0=0x00 R1=0x00 R2=0x00 R3=0x00 R4=0x00 R5=0x00 R6=0x00 R7=0x00 flags=---- Faulted (StackOverflow) cycles=23040
```

Say this in the viva

"`operator<<` is a friend function of CPU. It can't be a member because the stream is on the left of `<<`. Being a friend, it reads the private registers directly and prints the whole CPU on one line. It returns the stream so it can be chained."

3.8 Friend class: CpuSnapshot

The idea in one line

A **friend class** gets access to all private members of another class. Used when two classes are designed to work closely together, so you don't have to make everything public.

CPU/Aashish/src/CpuSnapshot.h · lines 8–31

```
 8  //CpuSnapshot is a friend of CPU, so it copies the private members straight in and out: registers,
 9  //PC, SP, flags, state, the fault with all its details, cycles and the instruction trace.
10  //The public setters cannot do all of that (setState can only keep or clear a fault, never put one back),
11  //and adding a public setter for every member would let ANY code change them. A friend class gives
12  //that access to this one trusted class only.
13  //Memory is not part of it: undo memory with Bus::poke and the write listener's old values.
14  class CpuSnapshot{
15      public:
16          CpuSnapshot();                          //a CPU straight after reset()
17          explicit CpuSnapshot(const CPU& cpu);   //copies everything inside this CPU
18
19          void restore(CPU& cpu) const;           //puts all of it back
20
21      private:
22          Byte  reg[CPU::NUM_REGS];
23          Word  pc;
24          Word  sp;
25          Byte  flags;
26          CpuState state;
27          Fault fault;
28          unsigned long cycles;
29          Word  instrStart;
30          InstructionTrace trace;
31  };
```

CPU/Aashish/src/CpuSnapshot.cpp · lines 17–27

```
17  void CpuSnapshot::restore(CPU& cpu) const{
18      for(int i = 0; i < CPU::NUM_REGS; ++i) cpu.reg[i] = reg[i];
19      cpu.pc = pc;
20      cpu.sp = sp;
21      cpu.flags = flags;
22      cpu.state = state;
23      cpu.fault = fault;
24      cpu.cycles = cycles;
25      cpu.instrStart = instrStart;
26      cpu.trace = trace;
27  }
```

Why a friend and not setters? The CPU's public setters can't put a fault back: `setState` can only keep or clear one. There are also members with no setter at all (`instrStart`, `trace`). Making everything public would let **any** code change them. `friend class CpuSnapshot;` gives that access to **one** trusted class only, so encapsulation stays strong. This is called the **Memento** pattern. A test proves the friend can do what the setters can't:

CPU/Aashish/tests/test_cpu.cpp · lines 769–780

```
769       //the public setters cannot put a fault back (setState only keeps or clears one); the friend class
              can
770       Byte bad[] = {0x00, 0xFF};
771       RUN(m, bad);
772       Fault before = m.cpu.getFault();
773       CpuSnapshot crashed(m.cpu);
774       m.cpu.reset();
775       crashed.restore(m.cpu);
776       const Fault& after = m.cpu.getFault();
777       check(m.cpu.getState() == CpuState::Faulted && after.type == FaultType::InvalidOpcode && after.type
              == before.type
778           && after.pc == before.pc && after.address == before.address && after.message == before.message,
779           "restore brings back a fault with all its details");
780       check(m.cpu.step() == StepResult::Fault && m.cpu.getPC() == 0x0001, "and the CPU acts faulted
              again");
```

Say this in the viva

"`CpuSnapshot` is a friend class of CPU. It copies every private member out and back, including the fault details and the trace, which the public setters can't restore. Instead of making all of that public, `friend` gives access to this one class only, so it's a save point without weakening encapsulation."

3.9 Static members

CPU/Aashish/src/Opcodes.h · lines 47–52

```
47  class OpcodeTable{
48      public:
49          static const OpcodeInfo& lookup(Byte opcode);
50          static int  find(const std::string& mnemonic);      //opcode number, or -1
51          static Byte lengthOf(Format format);
52  };
```

- **Static member functions:** `OpcodeTable::lookup`, `find`, `lengthOf`. There is only one instruction set, so they are called on the class, `OpcodeTable::lookup(0x30)`, without creating an object.
- **Static constants:** `CPU::NUM_REGS` (`CPU.h:24`), `InstructionTrace::CAPACITY`, `Console::CYCLES_PER_FRAME`. One value shared by every object.
- **Function-local static:** the opcode table is built inside `table()` the first time it is called (`static const Table t;`). This avoids the "static initialisation order" problem, where a global could be used by another file before it exists.

3.10 Functions

Feature	Example in the CPU
Pass by reference (out-parameter)	`bool pop(Byte& value)`: returns success **and** gives back the value
Pass by `const` reference	`InstructionTrace& getTrace() const`, `CpuSnapshot(const CPU& cpu)`: no copy, read-only
Default arguments	`void reset(Word startAddress = 0x0000)`, `runFrame(unsigned long cycleBudget = CYCLES_PER_FRAME)`
`const` member functions	every getter, e.g. `Word getPC() const`: promises not to change the object
Return values for errors	`bool write(...)`, `StepResult step()`: expected errors are data, not exceptions
Helper functions in an anonymous namespace	`hex8`, `hex16` in `CPU.cpp`: visible only inside that file

3.11 Other C++ features worth knowing

Feature	Where	Why
`typedef`, fixed-width types	`Byte` = `uint8_t`, `Word` = `uint16_t` (`Bits.h`)	exact 8/16-bit sizes
`enum class`	`CpuState`, `StepResult`, `FaultType`, `Format`	scoped, can't be mixed up with ints
Enum with an underlying type	`enum Opcode : Byte`	opcodes are byte values
`std::function` + lambdas	`Bus::setWriteListener`, `IoPorts::setPpuListener`	other units plug in their own code (callbacks); this is how undo records memory writes
Bitwise operators	`Bits.cpp`, flags, `CPU::add`, RegReg decoding, the random-number LFSR	hardware works in bits
STL containers	`std::vector`, `std::unordered_map`, `std::string`	safe, automatic memory
Header guards	`#ifndef CPU_H` ... `#endif` in every header	each header is included once

Part 4 · How the CPU is tested

193 automatic checks, and how to run them.

The tests use a tiny home-made framework: one function that prints PASS or FAIL and counts them. No library needed.

CPU/Aashish/tests/test_cpu.cpp · lines 28–37

```
28  void check(bool condition, const char* name){
29      if(condition){
30          printf("[PASS] %s\n", name);
31          passed++;
32      }
33      else{
34          printf("[FAIL] %s\n", name);
35          failed++;
36      }
37  }
```

A typical test sets up a machine, runs a real program and checks the result. This one tests the circular queue inside the real CPU:

CPU/Aashish/tests/test_cpu.cpp · lines 672–685

```
672      //the CPU keeps one: after a fault it shows the way the program got there
673      Machine m;
674      Byte program[] = {
675          0x11, 0x00, 0x05,          //0x0000 LDI R0, 5
676          0x41, 0x00,                //0x0003 DEC R0      Loop
677          0x62, 0x03, 0x00,          //0x0005 JNZ 0x0003
678          0xFF                       //0x0008 not an instruction
679      };
680      RUN(m, program);
681      const InstructionTrace& t = m.cpu.getTrace();
682      check(t.size() == 12 && t.at(0).pc == 0x0000 && t.at(0).opcode == OP_LDI && t.at(1).opcode == OP_DEC,
683          "the CPU records every instruction it starts (1 LDI + 5 x DEC, JNZ + the bad byte)");
684      check(t.at(11).pc == 0x0008 && t.at(11).opcode == 0xFF && t.at(10).pc == 0x0005 && t.at(10).opcode
          == OP_JNZ,
685          "the failing instruction is the newest entry, the one before it shows how we got there");
```

Suite (folder CPU/)	Checks	What it covers
test_cpu (mine)	193	bits, RAM/ROM, Bus and its binary search, all 38 instructions, flags, stack, every fault, I/O ports, the trace queue, operator<<, CpuSnapshot, Console
test_parser	23	reading .asm text (Arnav)
test_operand_check	51	rejecting bad operands; SystemState (Piyush)
test_assembler	25	machine-code output, and running it on my CPU
test_calc	30	the calculator, including **all 65,536 input pairs** for + − & | ^, compared with C++
Total	**322**	**0 failing**

How to run (Windows, in the CPU folder): `mingw32-make test` runs all five suites. In CPU/Aashish: `mingw32-make test` (CPU only), `mingw32-make demo` (the guided demo), `mingw32-make live` (the live dashboard), `mingw32-make add A=25 B=17` (one addition, step by step).

> **Say this in the viva**
> "The CPU has 193 automatic checks, every instruction, flag and fault, and the whole project has 322, all passing. The calculator tests every one of the 65,536 input pairs for five operators against C++, so the arithmetic is checked exhaustively, not on a few examples."

Part 5 · Viva cheat sheet

What to say, and where to point.

Explain the CPU in 60 seconds

Say this (about 60 seconds)

"I built the CPU and memory of PIXEL-8, an 8-bit computer in software, in C++. It has 8 registers, a program counter, a stack pointer and four flags, and 38 instructions. Every step it fetches an opcode through the Bus, decodes it by indexing a 256-entry array, and executes it in a switch.

Memory is 64 KB split into ROM, RAM, video RAM, I/O ports and the stack. Each chip is a class derived from an abstract `Device`, and the Bus finds the right chip with a binary search over sorted address ranges.

From the DS syllabus: arrays for registers and the opcode table, an array-based stack for CALL/RET, a circular queue that records the last 16 instructions, binary search in the Bus, and a hash map from instruction names to opcodes for the assembler. From C++: the four pillars, constructors with initializer lists, a friend `operator<<`, and a friend class `CpuSnapshot` for save points.

Bad programs never crash it: they raise one of 7 faults. It has 193 tests, all passing."

Quick answers

If they ask...	Answer	Point at
Where is the stack?	Array-based stack in stack memory, SP = top. Push: write, SP--. Pop: SP++, read. Used by CALL/RET, PUSH/POP.	CPU.cpp:308–328
Where is a queue?	`InstructionTrace`: circular queue of the last 16 instructions; rear = (front + count) % 16.	InstructionTrace.cpp:9–26
Where is binary search?	`Bus::find` over chip ranges sorted by `attach()`. O(log n), runs on every memory access.	Bus.cpp:40–51
Where is a hash map?	`OpcodeTable::find`: `unordered_map` name → opcode, used by the assembler. Opcode → info is a direct-address array.	Opcodes.cpp:108–115
Where are arrays?	`reg[8]`, `slot[256]`, `items[16]`, memory `vector`s.	CPU.h:55
Why no linked list?	Nothing inserts in the middle; everything is fixed-size or indexed, and binary search needs random access.	Part 2.6
Abstraction?	`Device`: abstract class, pure virtual `read`/`write`/`size`/`name`.	Device.h:8–20
Inheritance?	`Ram`, `Rom`, `IoPorts : public Device`.	Ram.h:9
Polymorphism?	Bus calls `device->read()` through `Device*` (run time); overloaded `loadProgram`, `operator<<` (compile time).	Bus.cpp:53–60
Encapsulation?	All CPU state private; const getters; validating setters.	CPU.cpp:373–376
Friend function?	`operator<<`: stream on the left, so a non-member; friend to read private registers.	CPU.cpp:439–453
Friend class?	`CpuSnapshot` saves/restores every private member, the fault included; access for one class only.	CpuSnapshot.cpp:17–27
Constructors?	Initializer lists, `explicit CPU(Bus&)`, default `Fault()`, overloaded `CpuSnapshot`, virtual destructor.	Console.cpp:12–19
Static members?	`OpcodeTable::lookup/find`, `NUM_REGS`, `CAPACITY`, function-local static table.	Opcodes.h:47–52

If they ask...	Answer	Point at
How is an instruction run?	Fetch at PC (PC++), decode by array index, execute in a switch; faults stop it cleanly.	CPU.cpp:37–63
How is it tested?	193 checks in `test_cpu.cpp`; 322 across the project; all 65,536 input pairs for the calculator.	Part 4

Careful

If you are asked something you don't know, say what the code **does** do and where. "I didn't use X; here is what I used instead and why" is a strong answer when the reason is real. Don't claim a structure is somewhere it isn't: every claim in this guide can be checked by opening the file at the line given.